

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 7

Part 0 – Warm-up: Explore the Base City

Q0.1.

- Describe how many main roads there are in each direction.
There are 2 two main roads running North–South and two main roads running East–West.
- How many signalized intersections are currently implemented?
There is only one fully signalized intersection in the scene. At this central junction, traffic lights are placed on all four corners, providing red and green signals for both the North–South direction and the East–West direction.
- How do vehicles behave when the light turns red?
Vehicles approaching the intersection will slow down as they detect a red light in their direction. If they are close enough to the junction, they come to a complete stop before entering the intersection and wait until the signal turns green again, after which they accelerate and continue driving normally.
- How do pedestrians behave? Do they ever stop?
Pedestrians follow a simple scripted behavior where they walk continuously along their assigned path without reacting to traffic lights or vehicles. They do not stop at any point—in the base version of the simulation, pedestrians walk straight through the environment regardless of the traffic conditions.

Q0.2.

- **build_traffic_lights(self):** This function constructs the traffic-light meshes (red and green bulbs for both directions) and places them into the scene while storing references for later updates.
- **update(self):** This per-frame update function controls the simulation loop by updating the traffic-light timer, refreshing light states, and calling the vehicle and pedestrian update routines.

- **update_vehicles(self, dt, ns_green, ew_green):** This function updates all vehicle movements, determining whether each car should drive, slow down, or stop based on the current traffic-light state for its direction.
- **init_pedestrians(self):** This function creates pedestrian objects and sets up their initial positions and walking directions in the scene.
- **update_pedestrians(self, dt):** This function moves pedestrians forward each frame according to a fixed walking speed, without any interaction with traffic lights or vehicles.

Part 1 – Traffic Lights with Yellow Signal

Task 1 – Extend the State Machine

T1.1. , T1.2. ,T1.3.

```
def update(self):
    # Gently rotate the Label
    self.label.rotate_y(0.2 * self.delta_time)

    # ----- Sun: sunrise -> midday -> sunset -----
    day_speed = 0.08
    theta = (self.time * day_speed) % math.pi

    sun_x = 120.0 * math.cos(theta)          # east (+x) -> west (-x)
    sun_y = 10.0 + 70.0 * math.sin(theta)    # low at horizon, high at midday
    sun_z = -60.0                          # fixed distance in front

    self.sun.set_position([sun_x, sun_y, sun_z])

    # Directional light always points from sun to city center
    target = [0.0, 0.0, 0.0]
    dir_x = target[0] - sun_x
    dir_y = target[1] - sun_y
    dir_z = target[2] - sun_z
    self.sun_light.set_direction([dir_x, dir_y, dir_z])
```

```

# ----- Traffic lights state -----
cycle_time = self.time % self.traffic_cycle

cycle = self.time % self.traffic_cycle
ns_green = ns_yellow = False
ew_green = ew_yellow = False

.....t1 = self.t_ns_green
t2 = t1 + self.t_ns_yellow
t3 = t2 + self.t_ew_green
t4 = t3 + self.t_ew_yellow

if cycle_time < t1:
    # (a) NS green, EW red
    ns_green = True
elif cycle_time < t2:
    # (b) NS yellow, EW red
    ns_yellow = True
elif cycle_time < t3:

```

```

# (c) NS red, EW green
ew_green = True
else:
    # (d) NS red, EW yellow
    ew_yellow = True

red_on = [1.0, 0.1, 0.1]
red_off = [0.2, 0.0, 0.0]
green_on = [0.2, 1.0, 0.2]
green_off = [0.0, 0.2, 0.0]
yellow_on = [1.0, 1.0, 0.2]
yellow_off = [0.2, 0.2, 0.0]

# update bulb colors via set_properties (material is read-only)
for m in self.ns_red_bulbs:
    m.material.set_properties({
        "baseColor": red_on if not (ns_green or ns_yellow) else red_off
    })

for m in self.ns_yellow_bulbs:
    m.material.set_properties({
        "baseColor": yellow_on if ns_yellow else yellow_off
    })

```

```

    })

    for m in self.ns_green_bulbs:
        m.material.set_properties({
            "baseColor": green_on if ns_green else green_off
        })

    # EW bulbs
    for m in self.ew_red_bulbs:
        m.material.set_properties({
            "baseColor": red_on if not (ew_green or ew_yellow) else red_off
        })

    for m in self.ew_yellow_bulbs:
        m.material.set_properties({
            "baseColor": yellow_on if ew_yellow else yellow_off
        })

    for m in self.ew_green_bulbs:
        m.material.set_properties({
            "baseColor": green_on if ew_green else green_off

```

```

            "baseColor": green_on if ew_green else green_off
        })

    # ----- Camera, vehicles, pedestrians -----
    smoothed_dt = min(self.delta_time, 0.033) # ~max step = 1/30 s
    self.update_vehicles(smoothed_dt, ns_green, ew_green)
    self.update_pedestrians(smoothed_dt)
    self.rig.update(self.input, smoothed_dt)

    # Render main camera (you can re-enable mini-map if you want)
    # self.renderer.render(self.scene, self.sky_camera)
    self.renderer.render(self.scene, self.camera)

```



T1.4.

- Yes. The bulbs follow the sequence: green->Yellow->Red
- No. As the code below, the situation won't happen.

```

929         t1 = self.t_ns_green
930         t2 = t1 + self.t_ns_yellow
931         t3 = t2 + self.t_ew_green
932         t4 = t3 + self.t_ew_yellow
933
934         if cycle_time < t1:
935             # (a) NS green, EW red
936             ns_green = True
937         elif cycle_time < t2:
938             # (b) NS yellow, EW red
939             ns_yellow = True
940         elif cycle_time < t3:
941             # (c) NS red, EW green
942             ew_green = True
943         else:
944             # (d) NS red, EW yellow
945             ew_yellow = True

```

Part 2 – More Realistic Vehicle Behavior at Intersections

Task 2– Stop Line and “Has Passed” Flag

Part 3 – Traffic Lights at All Intersections

Task 3 – Replicate and Manage Multiple Intersections

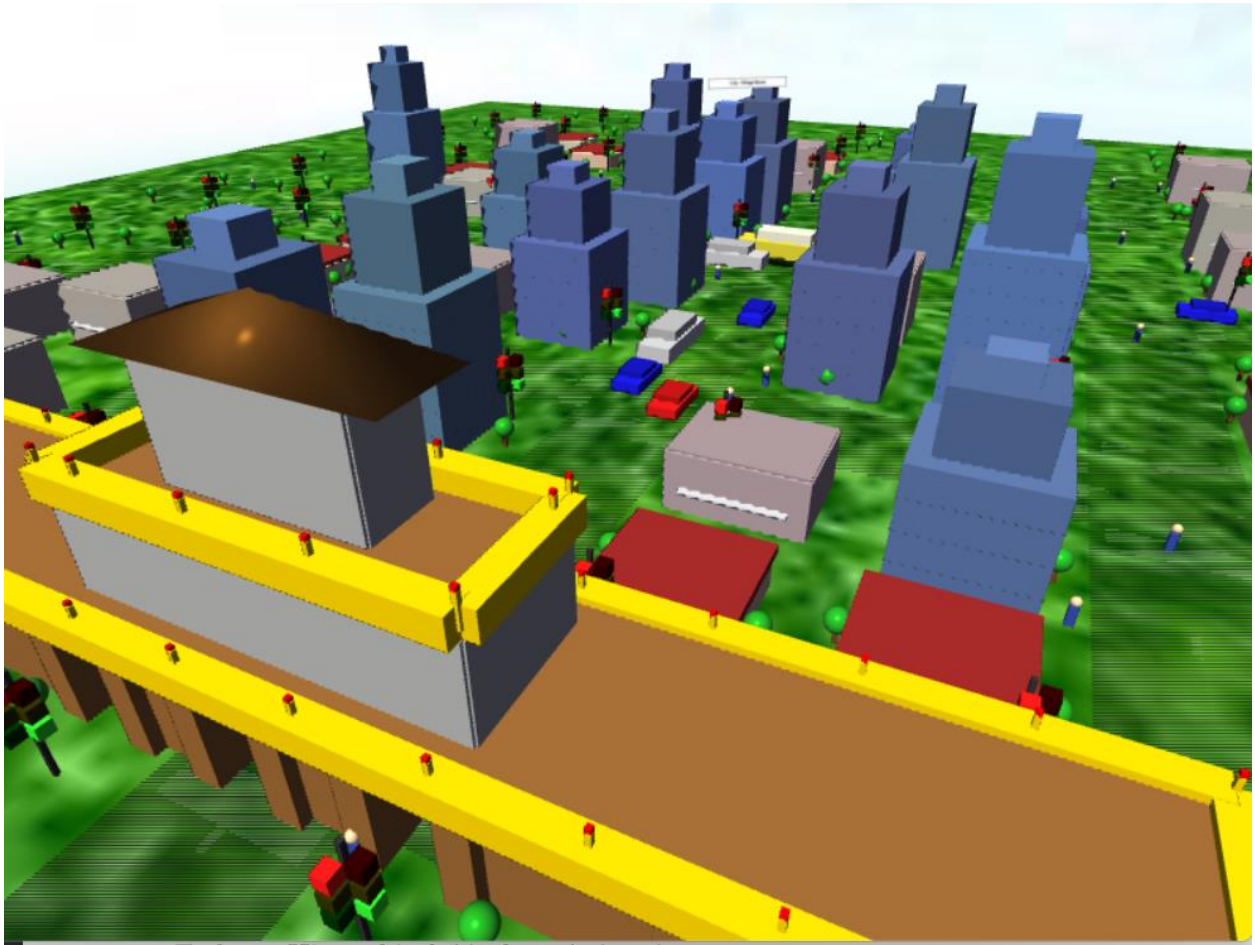
Part 4 – Smarter Pedestrians

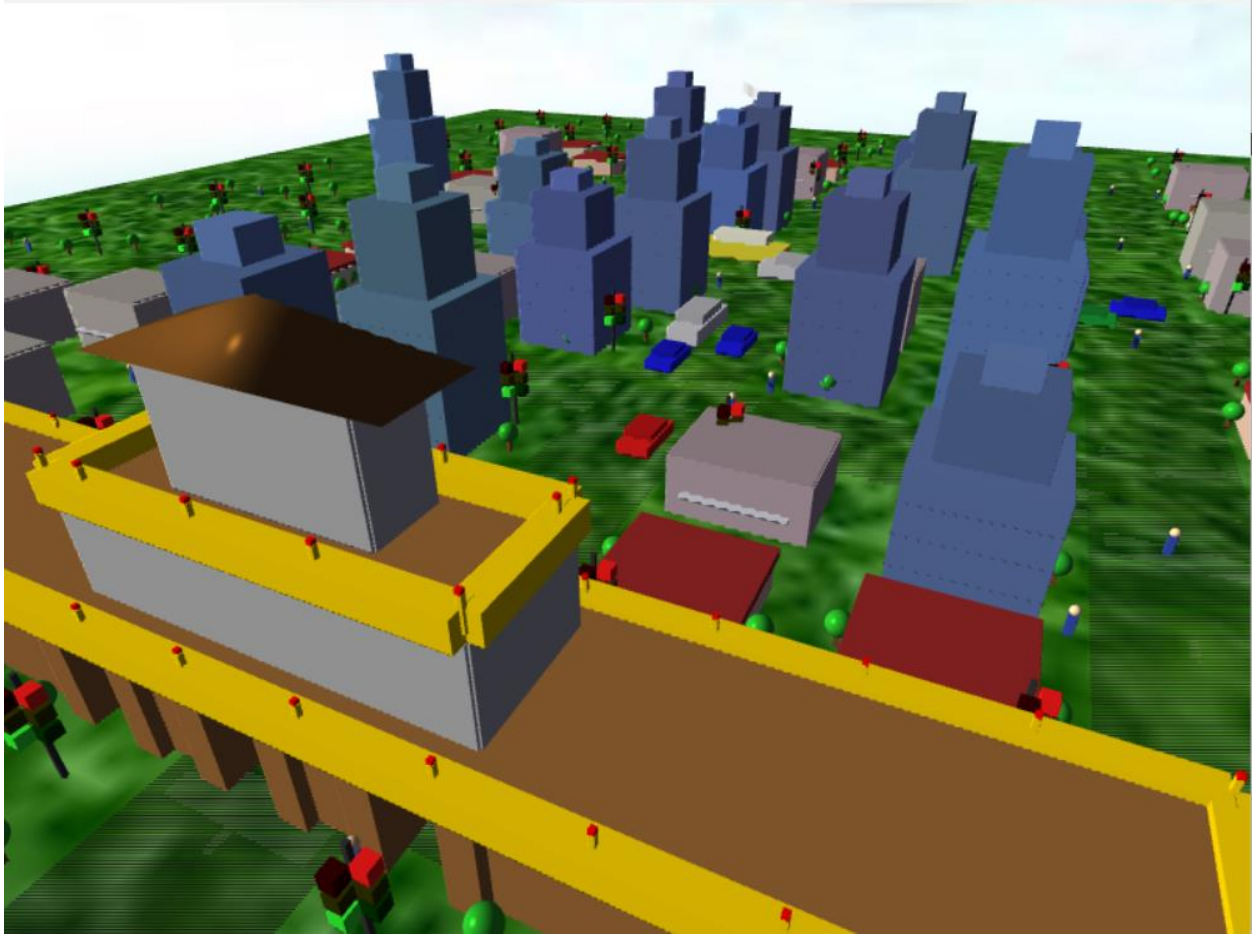
Task 4 – Crossing Logic

Part 5 – Integrating a Landmark from Lab 6

Task 5 – Reuse and Place Your Previous Scene

Long afternoon:





Part 6 – Airplane Flying Over the City

Task 6 – Hierarchical Airplane Animation


```

646 # ----- helper: airplane -----
647 def make_airplane(self):
648     # Create Root Node
649     self.plane_root = Object3D()
650
651     # Material: White/Silver so shadows show up nicely
652     plane_mat = SurfaceMaterial(property_dict={
653         "baseColor": [0.9, 0.9, 1.0],
654         "doubleSide": True
655     })
656
657     # --- 1. Fuselage (The Body) ---
658     # Long along the Z axis
659     fuselage = Mesh(BoxGeometry(width=1, height=1, depth=6), plane_mat)
660     self.plane_root.add(fuselage)
661
662     # --- 2. Main Wings ---
663     # Wide along X axis
664     wings = Mesh(BoxGeometry(width=8, height=0.2, depth=2), plane_mat)
665     # Move wings slightly forward (negative Z is usually "forward" in OpenGL)
666     wings.set_position([0, 0, -1])
667     self.plane_root.add(wings)
668
669     # --- 3. Vertical Tail ---
670     # Tall, at the back (positive Z)
671     tail_v = Mesh(BoxGeometry(width=0.4, height=2, depth=1.5), plane_mat)
672     tail_v.set_position([0, 1, 2.5])
673     self.plane_root.add(tail_v)

```

```

# --- 4. Horizontal Tail ---
# Wide, at the back
tail_h = Mesh(BoxGeometry(width=3, height=0.2, depth=1.5), plane_mat)
tail_h.set_position([0, 0.5, 2.5])
self.plane_root.add(tail_h)

# Add the whole group to the main scene
self.scene.add(self.plane_root)

```